

VITyarthi - Build Your Own Project

School Management System

A modular, object-oriented Python application for managing students, teachers and grades

Course Name & Code	CSE1021 - Introduction to Problem Solving and Programming
Student Name	Rudraksh Borde
Registration Number	26BAI10119
Submission Date	29 September 2026

Project Report

Table of Contents

1. Cover Page
2. Introduction
3. Problem Statement
4. Functional Requirements
5. Non-functional Requirements
6. System Architecture
7. Design Diagrams
8. Design Decisions & Rationale
9. Implementation Details
10. Screenshots / Results
11. Testing Approach
12. Challenges Faced
13. Learnings & Key Takeaways
14. Future Enhancements
15. References

2. Introduction

Every school, however small, must keep track of the people who make it work: the students who enrol, the teachers who teach them, and the marks the students earn. When these records are kept on paper or in scattered files, mistakes are easy to make and information is hard to find.

The **School Management System** is a console application written in Python that keeps this information in one structured place. A user chooses an option from a numbered menu, answers a few prompts, and the system validates the input, updates the record and saves it permanently to a JSON file. The next time the program is started, all earlier records are available again.

The project applies the core concepts of object-oriented programming: **abstraction** (an abstract base class `persons`), **inheritance** (`Student` and `Teacher` extend it), **polymorphism** (each subclass implements `register()` and `show_details()` in its own way) and **encapsulation** (storage, business logic and menu are separate modules). It also uses file handling, JSON serialisation, input validation and automated unit testing.

3. Problem Statement

Small schools and coaching centres often store student, teacher and grade records manually. This leads to three recurring problems:

- **Duplicate or inconsistent entries** - the same roll number or employee ID being issued twice, or mistyped email addresses being stored.
- **Slow retrieval** - finding one student's details and grades means searching through paper or files.
- **No reliable storage** - information is not kept in a single, consistent, persistent place.

The problem is to design and build a simple, lightweight system that records students, teachers and grades in a structured way, rejects invalid or duplicate data at the moment of entry, and stores everything permanently so it can be retrieved quickly at any time.

Objectives

- Provide a menu-driven interface for registering students and teachers.
- Validate email addresses and enforce unique roll numbers and employee IDs.
- Allow subject-wise grades to be recorded against a student.
- Allow the full details of a student or teacher to be displayed on request.
- Persist all data in a JSON file so it survives between runs.
- Apply OOP principles and a modular structure, and verify behaviour with automated tests.

4. Functional Requirements

The system is organised into **three major functional modules**, each covering a group of related features.

ID	Module	Requirement	Input	Output
FR1	Student management	Register a student with name, age, email and roll number.	Name, age, email, roll number	Confirmation, record saved

ID	Module	Requirement	Input	Output
FR2	Student management	Reject a student whose email is invalid or whose roll number already exists.	Email, roll number	Error message; nothing saved
FR3	Teacher management	Register a teacher with name, age, email, subject and employee ID.	Name, age, email, subject, employee ID	Confirmation, record saved
FR4	Teacher management	Reject a teacher whose email is invalid or whose employee ID already exists.	Email, employee ID	Error message; nothing saved
FR5	Grades & records	Add a grade (subject + grade) to an existing student; several grades allowed.	Roll number, subject, grade	Confirmation, grade saved
FR6	Grades & records	Report 'Student not found' when a grade is added for an unknown roll number.	Roll number	Error message
FR7	Grades & records	Display a student's profile including all grades.	Roll number	Name, age, email, roll number, grades
FR8	Grades & records	Display a teacher's profile.	Employee ID	Name, age, email, subject, employee ID
FR9	Cross-cutting	Save every change to a JSON file and reload it at start-up.	-	School_data.json

Input / output structure and user workflow

Input is entered at the keyboard in response to text prompts; output is printed to the console. The user workflow is: (1) start the program, (2) read the menu of five options, (3) type the option number, (4) answer the prompts for that option, (5) read the confirmation or error message. The full flow is shown in the workflow diagram in Section 7.

5. Non-functional Requirements

ID	Category	Requirement	How it is met
NFR1	Usability	A first-time user must be able to operate the system without training.	Numbered five-option menu; plain-language prompts and confirmation / error messages.
NFR2	Reliability / persistence	Data must not be lost when the program closes.	Every add operation calls save(), which writes the complete data to School_data.json; the file is reloaded at start-up.
NFR3	Data integrity / validation	Invalid and duplicate records must not be stored.	validate_email() check plus duplicate roll-number / employee-ID search before any record is appended.

ID	Category	Requirement	How it is met
NFR4	Maintainability	Code should be easy to understand, change and extend.	Seven small modules with one responsibility each; abstract base class defines the contract for new person types; docstrings on each module.
NFR5	Portability	Must run on any common operating system with minimal setup.	Pure Python 3.8+ using only the standard library; no external dependencies.
NFR6	Testability	Behaviour must be verifiable automatically.	25 unit tests using unittest and mock input; tests use a temporary file so real data is never touched.
NFR7	Resource efficiency	Should run comfortably on modest hardware.	Data held in one in-memory dictionary and one small JSON file; suited to the small datasets the project targets.

6. System Architecture

The system follows a simple **three-layer architecture** that separates what the user sees, the rules of the system, and where data is kept.

- **Presentation layer** - `main.py` starts the program and `menu.py` shows the menu and routes the user's choice.
- **Business-logic layer** - `persons.py` defines the abstract base class and email validation; `student.py` and `teacher.py` contain the registration, grading and lookup rules.
- **Data layer** - `storage.py` loads the JSON file into memory and provides `save()`; `School_data.json` is the persistent store.

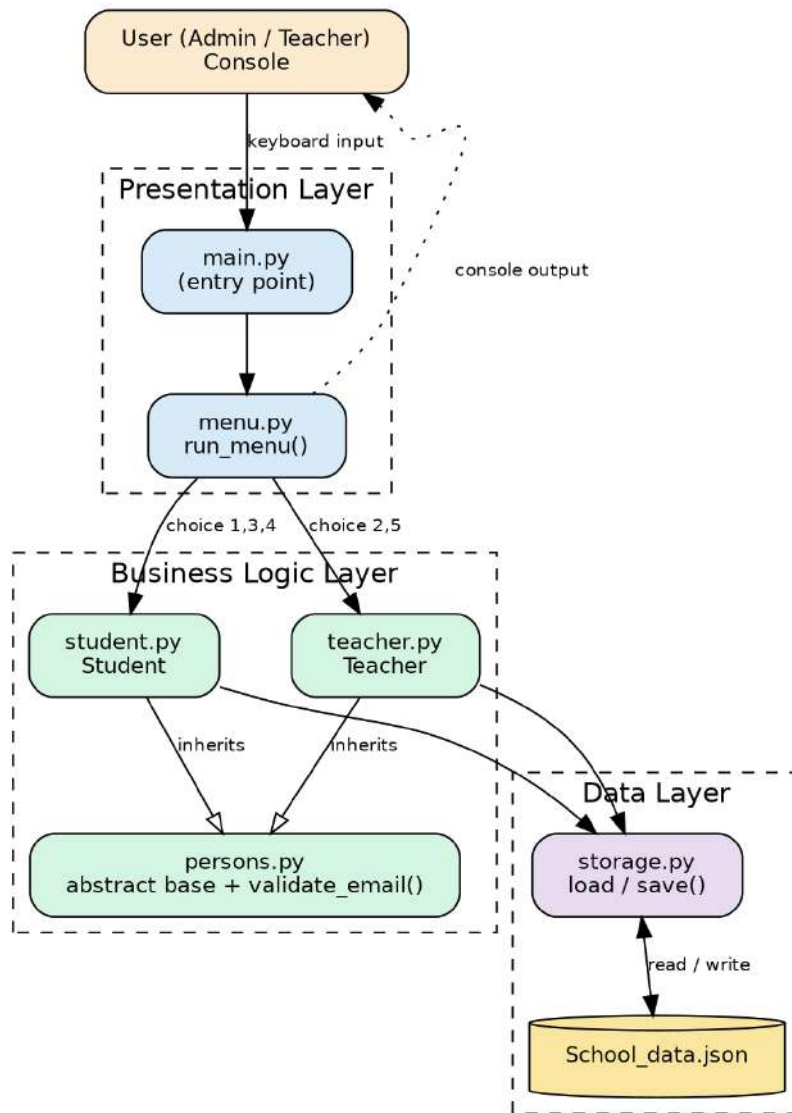


Figure 1: System architecture diagram

7. Design Diagrams

7.1 Use Case Diagram

A single actor (school administrator or teacher) interacts with five use cases. Registration use cases include email validation, duplicate-ID checking and saving; grade entry includes saving.

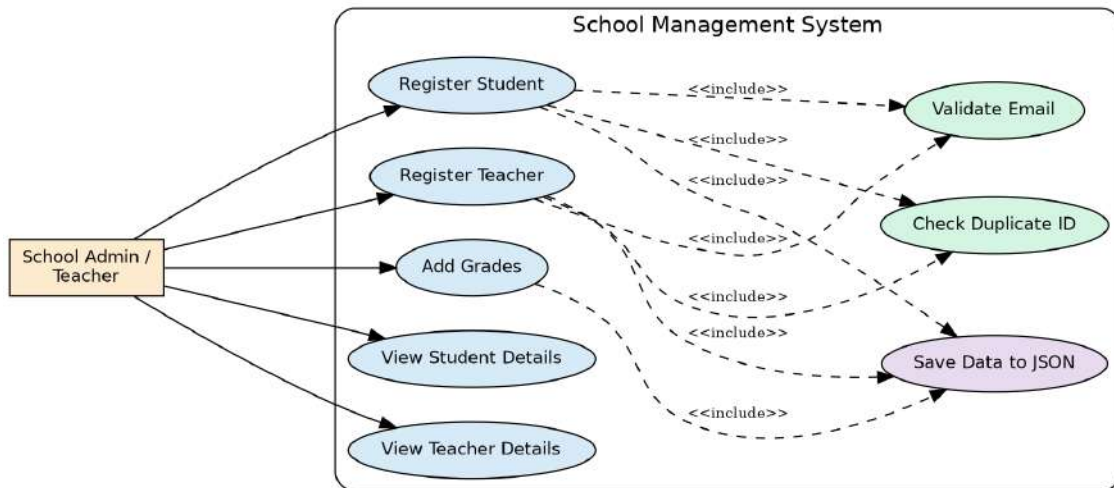


Figure 2: Use case diagram

7.2 Workflow Diagram

The workflow shows the path from start-up through the menu choice to validation, saving and output.

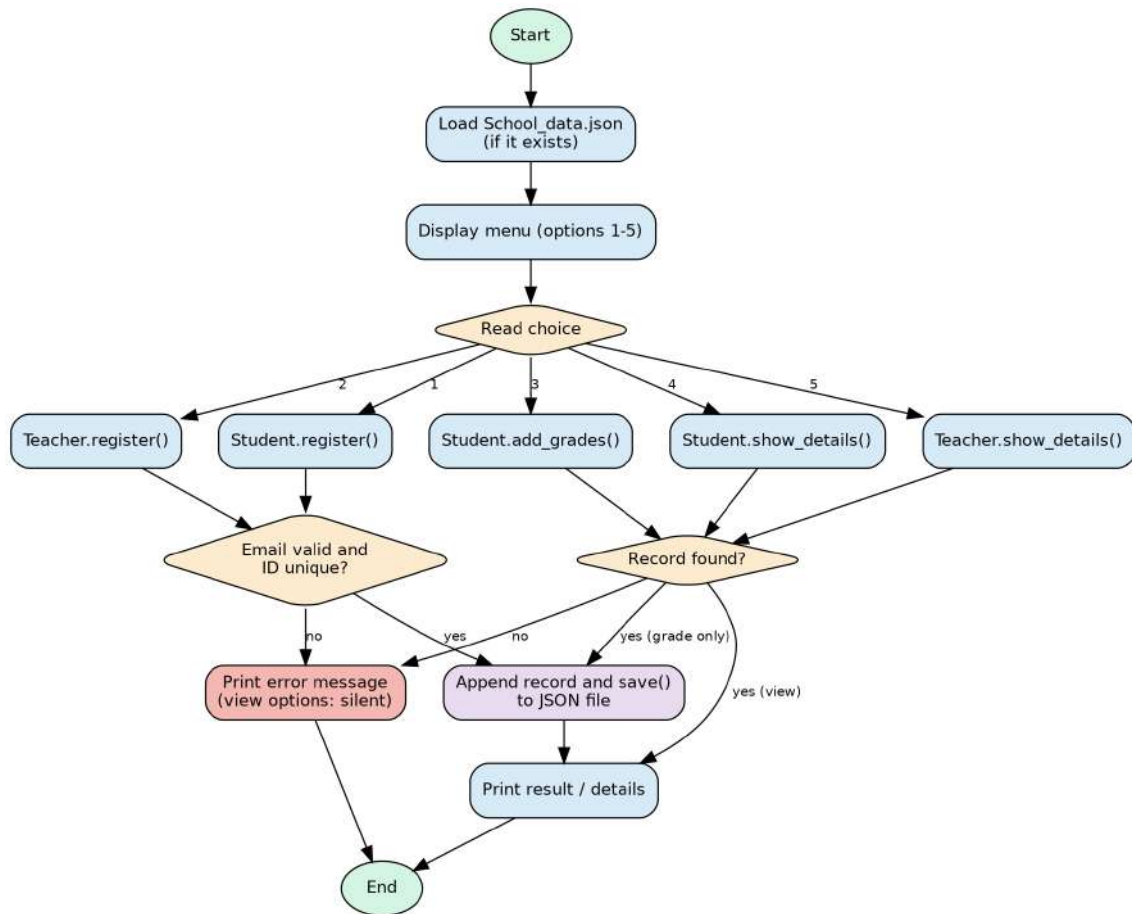


Figure 3: Workflow diagram

7.3 Sequence Diagram

The sequence below traces option 1 (register a student) through the modules until the record is stored.

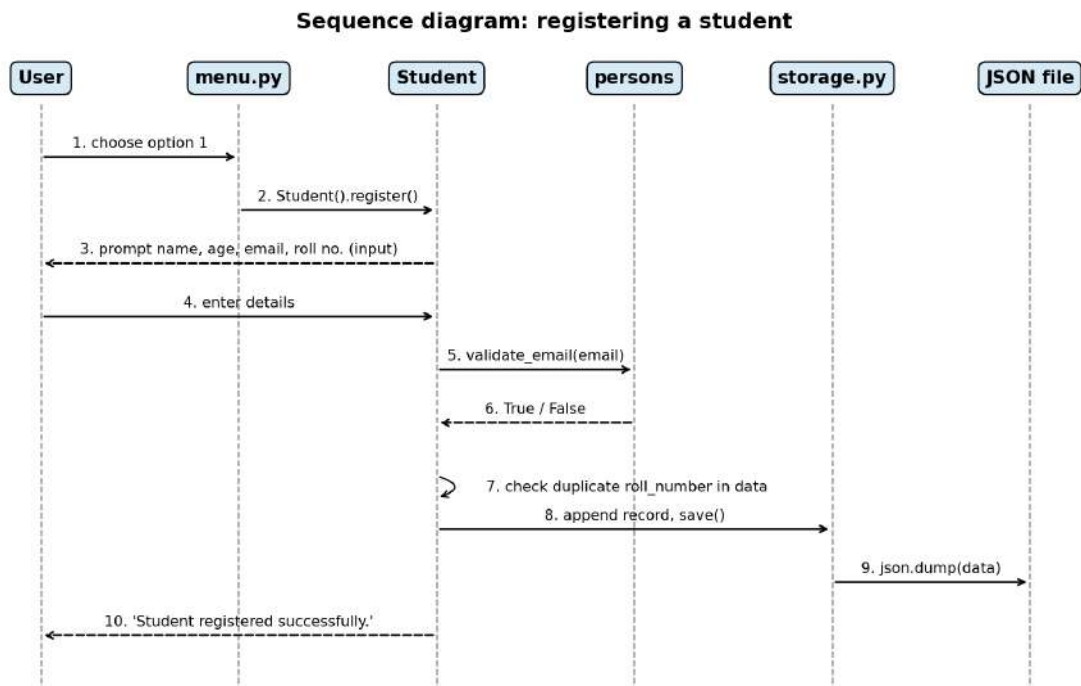


Figure 4: Sequence diagram - registering a student

7.4 Class Diagram

persons is abstract; Student and Teacher inherit from it and implement its abstract methods. Both use the shared data and save () from the storage module, and the menu creates both objects.

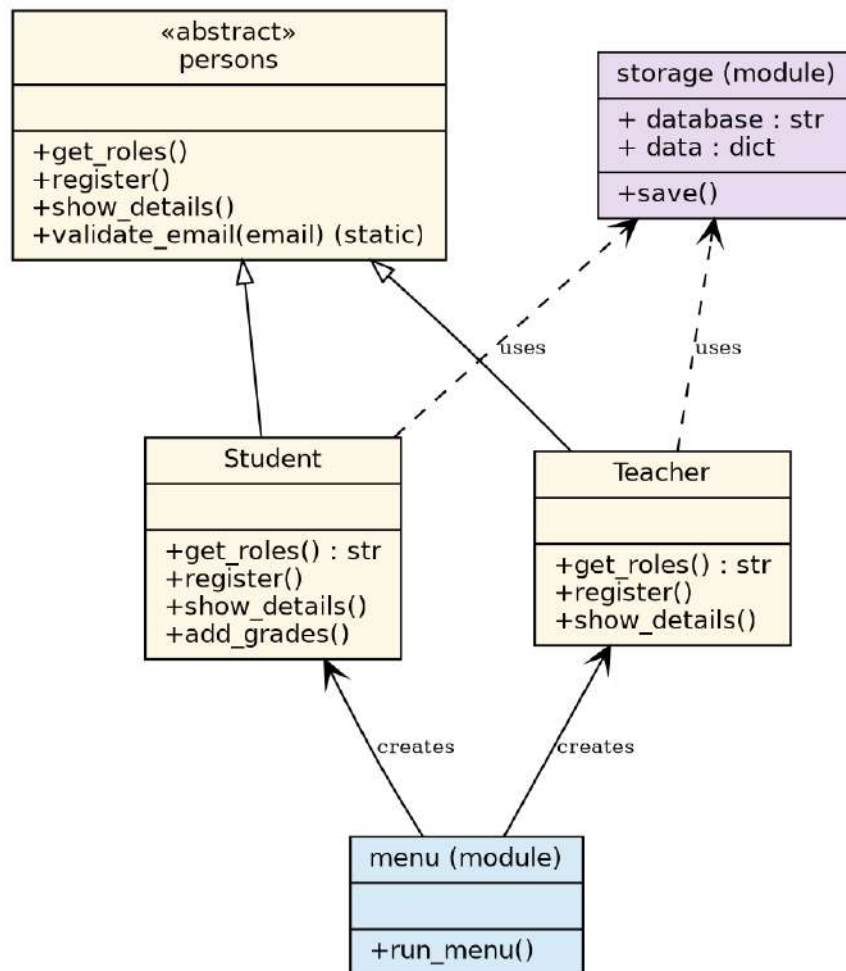


Figure 5: Class / component diagram

7.5 ER Diagram and Storage Schema

Data is stored as one JSON document with two top-level lists. Grades are nested inside each student, which is the JSON equivalent of a one-to-many relationship.

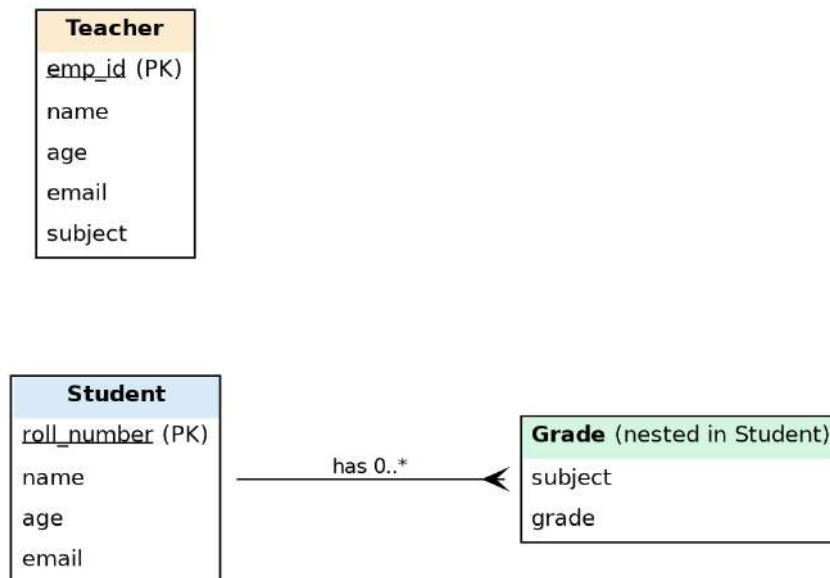


Figure 6: Entity-relationship diagram

```
{
  "Students": [
    {
      "name": "Asha Rao", "age": 16, "email": "asha@school.edu",
      "roll_number": "S101",
      "grades": [ { "subject": "Physics", "grade": "A" } ]
    },
    {
      "name": "Ravi Kumar", "age": 41, "email": "ravi@school.edu",
      "subject": "Physics", "emp_id": "T201"
    }
  ],
  "Teachers": [
    {
      "name": "Ravi Kumar", "age": 41, "email": "ravi@school.edu",
      "subject": "Physics", "emp_id": "T201"
    }
  ]
}
```

Schema notes: roll_number and emp_id are unique keys, enforced by a duplicate check in code; age is an integer; all other fields are strings.

8. Design Decisions & Rationale

Decision	Rationale
Abstract base class persons	Students and teachers share behaviour (registering, showing details, email validation). An abstract class forces every person type to implement the same methods and holds shared code in one place.
Static validate_email in the base class	Email validation does not depend on any one object, so a static method lets both subclasses reuse it without duplication.
JSON file instead of a database server	The data is small and hierarchical (grades belong to students). JSON is human-readable, needs no installation and is part of Python's standard library.
Single shared data dictionary in storage.py	Loading once at start-up and saving after each change keeps the code simple. All modules import the same dictionary object, so they always see the same data.

Decision	Rationale
Package split into 7 modules	Separating menu, entities and storage gives each file one job, makes testing easier and satisfies the modular-structure requirement.
Original logic left unchanged when modularising	The program was reorganised, not rewritten: the class bodies were moved into modules verbatim (only import lines and a run_menu() wrapper were added). This limits the risk of introducing new bugs.
unittest with mocked input	It is built into Python (no installation), and mocking input () lets the interactive code be tested automatically.

9. Implementation Details

9.1 Module overview

File	Responsibility
main.py	Entry point; calls run_menu().
school_mgmt/menu.py	Prints the five options, reads the choice and calls the matching method.
school_mgmt/persons.py	Abstract base class persons (get_roles, register, show_details) and the static validate_email().
school_mgmt/student.py	Student: register(), add_grades(), show_details().
school_mgmt/teacher.py	Teacher: register(), show_details().
school_mgmt/storage.py	Loads School_data.json into the data dictionary; save() writes it back.
tests/test_school.py	25 unit tests.

9.2 Key code

Abstract base class and validation (persons.py):

```
class persons(ABC):
    @abstractmethod
    def get_roles(self): pass
    @abstractmethod
    def register(self): pass
    @abstractmethod
    def show_details(self): pass

    @staticmethod
    def validate_email(email):
        if "@" in email and "." in email:
            return True
        return False
```

Registration with validation and duplicate check (student.py):

```
if not persons.validate_email(email):
    print("Email is invalid.")
    return

for i in data['Students']:
    if i['roll_number'] == roll_number:
        print("Roll number already exists.")
```

```
        return

data['Students'].append({... "grades": []})
save()
print("Student registered successfully.")
```

Persistence (storage.py): the file is read once when the module is imported, and save() rewrites it after every change:

```
if Path(database).exists():
    with open(database, "r") as f:
        content = f.read()
        if content:
            data = json.loads(content)

def save():
    with open(database, "w") as f:
        json.dump(data, f, indent=4)
```

9.3 Version control

The project is managed with Git and hosted on GitHub. The repository contains the source package, tests, documentation (README.md, statement.md), design diagrams, screenshots and this report.

10. Screenshots / Results

The screenshots below were produced by running the actual program (`main.py`) with scripted input. Each run is a separate launch of the program; Figure 12 shows the data file that accumulates across those runs, which demonstrates persistence.

```
$ python main.py # option 1

Press 1 to register a Student
Press 2 to register a Teacher
Press 3 to add Grades
Press 4 to show a Student details
Press 5 to show a teacher details
Enter your choice: 1
Enter Student Name: Asha Rao
Enter Student Age: 16
Enter Student Email: asha@school.edu
Enter Student Roll Number: S101
Student registered successfully.
```

Figure 7: Registering a student (option 1)

```
$ python main.py # option 2

Press 1 to register a Student
Press 2 to register a Teacher
Press 3 to add Grades
Press 4 to show a Student details
Press 5 to show a teacher details
Enter your choice: 2
Enter Student Name: Ravi Kumar
Enter Student Age: 41
Enter Student Email: ravi@school.edu
Enter Subject: Physics
Enter Employee ID: T201
Teacher registered successfully.
```

Figure 8: Registering a teacher (option 2)

```
$ python main.py # option 3

Press 1 to register a Student
Press 2 to register a Teacher
Press 3 to add Grades
Press 4 to show a Student details
Press 5 to show a teacher details
Enter your choice: 3
Enter Student Roll Number: S101
Enter Subject: Physics
Enter Grade: A
Grade added successfully.
```

Figure 9: Adding a grade to a student (option 3)

```
$ python main.py # option 4

Press 1 to register a Student
Press 2 to register a Teacher
Press 3 to add Grades
Press 4 to show a Student details
Press 5 to show a teacher details
Enter your choice: 4
Enter Student Roll Number: S101
Name: Asha Rao
Age: 16
Email: asha@school.edu
Roll Number: S101
Grades:
Physics: A
```

Figure 10: Showing student details with grades (option 4)

```
$ python main.py # option 5

Press 1 to register a Student
Press 2 to register a Teacher
Press 3 to add Grades
Press 4 to show a Student details
Press 5 to show a teacher details
Enter your choice: 5
Enter Employee ID: T201
Name: Ravi Kumar
Age: 41
Email: ravi@school.edu
Subject: Physics
Employee ID: T201
```

Figure 11: Showing teacher details (option 5)

```

$ cat School_data.json

{
  "Students": [
    {
      "name": "Asha Rao",
      "age": 16,
      "email": "asha@school.edu",
      "roll_number": "S101",
      "grades": [
        {
          "subject": "Physics",
          "grade": "A"
        }
      ]
    }
  ],
  "Teachers": [
    {
      "name": "Ravi Kumar",
      "age": 41,
      "email": "ravi@school.edu",
      "subject": "Physics",
      "emp_id": "T201"
    }
  ]
}

```

Figure 12: School_data.json after the runs above (persistent storage)

```

$ python main.py # duplicate roll number

Press 1 to register a Student
Press 2 to register a Teacher
Press 3 to add Grades
Press 4 to show a Student details
Press 5 to show a teacher details
Enter your choice: 1
Enter Student Name: Meena
Enter Student Age: 17
Enter Student Email: meena@school.edu
Enter Student Roll Number: S101
Roll number already exists.

```

Figure 13: Duplicate roll number rejected

```

$ python main.py # invalid email

Press 1 to register a Student
Press 2 to register a Teacher
Press 3 to add Grades
Press 4 to show a Student details
Press 5 to show a teacher details
Enter your choice: 1
Enter Student Name: Meena
Enter Student Age: 17
Enter Student Email: meena-at-school
Enter Student Roll Number: S102
Email is invalid.

```

Figure 14: Invalid email rejected

```

$ python main.py # grade for unknown student

Press 1 to register a Student
Press 2 to register a Teacher
Press 3 to add Grades
Press 4 to show a Student details
Press 5 to show a teacher details
Enter your choice: 3
Enter Student Roll Number: S999
Student not found.

```

Figure 15: Grade for an unknown student

11. Testing Approach

Testing uses Python's built-in unittest framework. Because the program reads user input, `unittest.mock.patch` replaces `input()` with a scripted list of answers, and printed output is captured to check messages. Each test points the storage module at a **temporary file** and clears the in-memory data first, so tests are independent and never modify real records.

Test class	Tests	What is verified
TestEmailValidation	4	Valid email accepted; missing '@', missing '.', empty string rejected.
TestAbstractBase	2	persons cannot be instantiated; <code>get_roles()</code> returns 'Student' / 'Teacher'.
TestStudent	9	Registration success and persistence to disk; invalid email; duplicate roll number; non-numeric age raises <code>ValueError</code> ; add one/multiple grades; unknown student; details output.
TestTeacher	4	Registration success and persistence; invalid email; duplicate employee ID; details output.
TestMenu	6	Options 1-5 route to the correct action; menu lists all five options.
Total	25	All 25 pass

How to run: `python -m unittest discover -s tests -v` from the project root. The result of the latest run is shown in Figure 16.

```
$ python -m unittest discover -s tests -v

test_persons_cannot_be_instantiated ... ok
test_roles ... ok
test_empty_string ... ok
test_missing_at_symbol ... ok
test_missing_dot ... ok
test_valid_email ... ok
test_choice_1_registers_student ... ok
test_choice_2_registers_teacher ... ok
test_choice_3_adds_grade ... ok
test_choice_4_shows_student ... ok
test_choice_5_shows_teacher ... ok
test_menu_lists_all_options ... ok
test_add_grade_success ... ok
test_add_grade_unknown_student ... ok
test_add_multiple_grades ... ok
test_register_duplicate_roll_number ... ok
test_register_invalid_email ... ok
test_register_non_numeric_age_raises ... ok
test_register_persists_to_disk ... ok
test_register_success ... ok
test_show_details ... ok
test_register_duplicate_emp_id ... ok
test_register_invalid_email ... ok
test_register_success ... ok
test_show_details ... ok

-----
Ran 25 tests in 0.018s

OK
```

Figure 16: Unit test results (25 tests, all passing)

In addition, manual validation tests were performed through the console (Figures 13-15): a duplicate roll number, a malformed email and an unknown student were all handled as designed. The test `test_register_non_numeric_age_raises` documents a known limitation, described in Section 14.

12. Challenges Faced

Challenge	How it was addressed
Splitting one script into several modules without changing behaviour	The class bodies were moved verbatim into their own files. A script compared every original line against the new package to confirm nothing had been altered; only imports, docstrings and a <code>run_menu()</code> wrapper were added.
Sharing one data dictionary across modules	If a module re-assigned the dictionary, other modules would keep a stale copy. All modules import the same object from <code>storage.py</code> and only modify it in place (append / item assignment).
Testing code that asks for keyboard input	Mocked <code>input()</code> with scripted answers and captured <code>stdout</code> , so the interactive flow could be tested without a human.

Challenge	How it was addressed
Keeping tests from overwriting real data	The storage file name is a module variable, so tests patch it to a temporary file and clear the in-memory lists between tests.
Deciding how far to go with validation	Email checking was kept simple ('@' and '.' present) to stay within scope; stronger validation and error handling for non-numeric input are listed as future work.

13. Learnings & Key Takeaways

- How abstract base classes enforce a common interface, and how inheritance removes duplicated code.
- How to separate a program into presentation, logic and data layers, and why small single-purpose modules are easier to test and maintain.
- How Python's JSON module and file handling provide simple, dependable persistence.
- How to test interactive programs using mocking and temporary files.
- How design documentation (use case, workflow, sequence, class and ER diagrams) clarifies a design before and after it is built.
- That honest documentation of limitations is part of good engineering: writing a test for the non-numeric-age case made the gap visible and easy to plan for.

14. Future Enhancements

- **Robust input handling** - wrap numeric input in try/except and re-prompt instead of raising ValueError; show a message when a student or teacher is not found.
- **Continuous menu** - keep the program running in a loop with an Exit option.
- **Edit and delete** - complete the CRUD operations for students, teachers and grades.
- **Better validation** - regular-expression email check, age range, numeric grade limits.
- **Reports** - class averages, grade sheets, top performers; export to CSV/PDF.
- **Logging** - record operations and errors in a log file for monitoring.
- **Security** - login with hashed passwords and role-based permissions.
- **Storage upgrade** - move from JSON to SQLite for larger datasets and querying.
- **Interface** - a Tkinter desktop GUI or web front end.
- **Wording fix** - correct the teacher registration prompts, which currently say 'Student'.

15. References

- [1] VITyarthi, *Build Your Own Project - General Project Instructions & Submission Guidelines*.
- [2] Python Software Foundation, *abc - Abstract Base Classes*.
<https://docs.python.org/3/library/abc.html>
- [3] Python Software Foundation, *json - JSON encoder and decoder*.
<https://docs.python.org/3/library/json.html>

- [4] Python Software Foundation, *pathlib - Object-oriented filesystem paths*.
<https://docs.python.org/3/library/pathlib.html>
- [5] Python Software Foundation, *unittest - Unit testing framework*.
<https://docs.python.org/3/library/unittest.html>
- [6] Python Software Foundation, *unittest.mock - mock object library*.
<https://docs.python.org/3/library/unittest.mock.html>
- [7] Graphviz, *DOT language and tools*. <https://graphviz.org/>
- [8] Hunter, J. D., *Matplotlib: A 2D graphics environment*, Computing in Science & Engineering, 2007.
- [9] ReportLab, *ReportLab PDF Toolkit User Guide*. <https://docs.reportlab.com/>